EE 66 Fall 2026 — Signals, Dynamics, and Information — Homework 6

This homework is due Friday, October 9, 2026, at 23:59.

Submission Format

Your homework submission should consist of a single PDF file that contains all of your answers (any handwritten answers should be scanned).

On occasion, a problem set contains one more problems designated as optional. Your solutions to such problems are not to be evaluated, and will not count toward your grade. Nevertheless, you are still responsible for learning the subject matter within their scope. In other words, we may test you on their scope.

1. 2D DTFS

In this problem, you will implement and use the 2D Discrete-Time Fourier Series (DTFS) on small images and then reconstruct the images using some of their DTFS coefficients. Use Python for all coding components.

You have seen one-dimensional time signals (i.e. audio waveforms). It turns out that we can think of images as signals - two-dimensional *spatial* signals, to be precise - too! An $N \times M$ image is a signal $x \in \mathbb{R}^{N \times M}$, indexed by row and column *position*, rather than time, indices.

For a signal $x \in \mathbb{R}^{N \times M}$, the 2D complex-exponential basis is indexed by (k, ℓ) and given by

$$\psi_{k,\ell}[n,m] = e^{i2\pi\left(\frac{kn}{N}+\frac{\ell m}{M}\right)}, \quad n = 0,\ldots,N-1,\ m = 0,\ldots,M-1.$$

They are orthogonal under the complex inner product and yield 2D DTFS analysis and synthesis equations:

$$X[k,\ell] = \frac{1}{NM}\sum_{n=0}^{N-1}\sum_{m=0}^{M-1} x[n,m]\, e^{-i2\pi\left(\frac{kn}{N}+\frac{\ell m}{M}\right)}, \qquad x[n,m] = \sum_{k=0}^{N-1}\sum_{\ell=0}^{M-1} X[k,\ell]\, e^{i2\pi\left(\frac{kn}{N}+\frac{\ell m}{M}\right)}.$$

This problem has an IPython component. Please attach a pdf printout of your IPython code to the HW in order to receive credit for problems that involve coding.

(a) Before writing any code, let's first work through one example manually. Take $N = M = 2$ in this part.

(i) Show that when $N = M = 2$, the basis images simplify to

$$\psi_{k,\ell}[n,m] = (-1)^{kn+\ell m},$$

so that every entry is either $+1$ or -1. Then write out each of $\psi_{0,0}, \psi_{0,1}, \psi_{1,0}, \psi_{1,1}$ as a 2×2 arrays.

(ii) Let $x = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$.
That is, $x[0,0] = 1$, $x[0,1] = 2$, $x[1,0] = 3$, and $x[1,1] = 4$. Using the analysis equation, compute all four coefficients $X[k,\ell]$.

Next part, you will implement the analysis equation for a general $N \times M$ image. The two sums you just expanded manually, over the rows and columns, will become nested loops to sum over n and m.

UCB EE 66, Fall 2026, Homework 6, All Rights Reserved. This may not be publicly shared without explicit permission.

Solution: **(i)** With $N = M = 2$,

$$\psi_{k,\ell}[n,m] = e^{i2\pi\left(\frac{kn}{2}+\frac{\ell m}{2}\right)} = e^{i\pi(kn+\ell m)} = \left(e^{i\pi}\right)^{kn+\ell m} = (-1)^{kn+\ell m},$$

using $e^{i\pi} = -1$. Since $kn+\ell m$ is an integer, each entry is $+1$ when $kn+\ell m$ is even and -1 when it is odd. Evaluating at $n,m \in \{0,1\}$:

$$\psi_{0,0} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \psi_{0,1} = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}, \qquad \psi_{1,0} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \qquad \psi_{1,1} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

Interpretation: $\psi_{0,0}$ is constant, $\psi_{0,1}$ represents change across the columns, $\psi_{1,0}$ represents change across the rows, and $\psi_{1,1}$ alternates in both directions at once to represent "diagonal" changes.

(ii) The analysis equation with $N = M = 2$ is

$$X[k,\ell] = \frac{1}{4}\sum_{n=0}^{1}\sum_{m=0}^{1} x[n,m]\, e^{-i2\pi\left(\frac{kn}{2}+\frac{\ell m}{2}\right)} = \frac{1}{4}\sum_{n=0}^{1}\sum_{m=0}^{1} x[n,m]\,(-1)^{kn+\ell m},$$

where we used $(-1)^{-t} = (-1)^t$. Writing out all four terms for $X[1,0]$, for instance,

$$X[1,0] = \frac{1}{4}\Big(\underbrace{x[0,0](-1)^0}_{1} + \underbrace{x[0,1](-1)^0}_{2} + \underbrace{x[1,0](-1)^1}_{-3} + \underbrace{x[1,1](-1)^1}_{-4}\Big) = \frac{1+2-3-4}{4} = -1.$$

Doing the same for the other three indices,

$$X[0,0] = \tfrac{1}{4}(1+2+3+4) = \tfrac{5}{2}, \qquad X[0,1] = \tfrac{1}{4}(1-2+3-4) = -\tfrac{1}{2},$$
$$X[1,0] = \tfrac{1}{4}(1+2-3-4) = -1, \qquad X[1,1] = \tfrac{1}{4}(1-2-3+4) = 0,$$

so

$$X = \begin{bmatrix} \frac{5}{2} & -\frac{1}{2} \\ -1 & 0 \end{bmatrix}.$$

Notice that $X[0,0] = \frac{5}{2}$ is the average of the four pixel values, which makes sense because $\psi_{0,0}$ is the constant image. This should remind you of our interpretation of the first 1D DTFS coefficient!

Also, all of the coefficients are real in this example; this is because when $N = M = 2$, the basis signals are real, and so is the image we are performing DTFS on. In general, this will *not* be true, when we write an arbitrary real signal as a sum of complex basis signals for other choices of N and M.

(b) Let x represent a grayscale image with N rows and M columns, so that $x[n,m]$ is the (real-valued) pixel in row n and column m.

In the iPython notebook, implement functions `dtfs(x)` and `inverse_dtfs(X)`, which carry out the analysis and synthesis equations above using nested loops, exactly as you expanded them in part (a). The notebook provides five 32×32 test images (horizontal stripes, vertical stripes, diagonal stripes, checkerboard, and dots). For each one, use your functions to compute X from x and then reconstruct $\widehat{x}$ from X, and report the largest error $\max_{n,m}|\widehat{x}[n,m] - x[n,m]|$ between the original and the reconstruction.

Solution: Implement the definitions exactly as written. Test correctness by confirming the reconstruction error is close to zero. See the solution IPython notebook for an example of a correct implementation.

(c) **In the iPython notebook**, implement function `visualize_dtfs_basis`, which draws the real part $\Re\{\psi_{k,\ell}\}$ of a basis image as a picture. Run it with $N = M = 32$ to display the basis images for every pair $k, \ell \in \{0, 3, 5\}$.

Looking at the resulting pictures, briefly describe how changing k affects the pattern compared to changing ℓ.

Solution: Increasing k increases vertical oscillations while increasing ℓ increases horizontal oscillations. For nonzero k and ℓ, the pattern mixes both directions. Wave patterns are along the 45 degree diagonal when $k = \ell$. See the solution IPython notebook for an example of a correct implementation of the basis functions.

(d) **In the iPython notebook**, implement function `verify_orthogonality`, which computes the inner product of two basis images,

$$\left\langle \psi_{k,\ell}, \psi_{k',\ell'} \right\rangle = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} e^{i2\pi\left(\frac{kn}{N}+\frac{\ell m}{M}\right)} e^{-i2\pi\left(\frac{k'n}{N}+\frac{\ell' m}{M}\right)}.$$

Run it on several pairs of indices with $(k,\ell) \neq (k',\ell')$, and also on a pair with $(k,\ell) = (k',\ell')$. Report the value you get in each of the two cases.

Solution: You should observe 0 (up to floating-point error) when $(k,\ell) \neq (k',\ell')$ and NM when $(k,\ell) = (k',\ell')$. See the solution IPython notebook for an example of a correct implementation of the inner product.

(e) **In the iPython notebook**, implement function `verify_parseval_theorem`, which computes the two quantities

$$\sum_{n=0}^{N-1} \sum_{m=0}^{M-1} |x[n,m]|^2 \qquad \text{and} \qquad NM \sum_{k=0}^{N-1} \sum_{\ell=0}^{M-1} |X[k,\ell]|^2.$$

The first adds up the energy of the image itself; the second adds up the energy of its DTFS coefficients. Run it on one of the test images and report both values along with their difference. These two quantities are always equal, a result known as Parseval's theorem. Your two numbers will differ by about 10^{-14}, which is only rounding in the computer's arithmetic.

Solution: Compute both quantities from your implementations; the relative error should be near machine precision for the test patterns. See the solution IPython notebook for an example of a correct implementation of both sides of the equation.

(f) For each provided image, plot the magnitude heatmap $|X[k,\ell]|$ (with labeled axes). Provide a brief description of what you see in each heatmap. The code for this part is provided.

Solution: Horizontal stripes produce a sequence of spikes with $\ell = 0$, vertical stripes produce a sequence of spikes with $k = 0$, diagonal stripes produce spikes along $k = \ell$, checkerboard produces a grid of varying-intensity spikes, and dots in time domain yield a grid of spikes in the frequency domain. See the solution IPython notebook for example heatmaps. Note that every signal has a DC offset (k=0, l=0) because the pixel values are always between 0 and 1.

(g) Consider the image compression problem: you want to store an image digitally in the most efficient manner possible. It turns out that DTFS is instrumental in solving this problem! In fact, the ubiquitous JPEG image compression format uses a very similar change-of-basis to DTFS. Suppose the image reconstructed by your compression strategy is $\widehat{x}$. Intuitively, this will in general *not* be identical to the original image x!

In this part, you will prove that keeping the largest DTFS coefficients is the best possible choice in the least squares sense.' This means that we are measuring the error by $\|x - \widehat{x}\|^2$.

For convenience, we relabel the NM index pairs (k,ℓ) with a single index j running from 1 to NM: that is, we list the pairs in some order as $(k_1,\ell_1),(k_2,\ell_2),\ldots,(k_{NM},\ell_{NM})$ and write ψ_j for ψ_{k_j,ℓ_j} and X_j for $X[k_j,\ell_j]$. The synthesis equation under this relabeling is

$$x = \sum_{j=1}^{NM} X_j \, \psi_j.$$

Suppose we choose to keep only B of the NM coefficients. Let $S \subseteq \{1,2,\ldots,NM\}$ be *any* subset of B indices, let c_j be *any* complex numbers such that $c_j = 0$ for $j \notin S$, and define the approximation

$$\widehat{x} = \sum_{j\in S} c_j \, \psi_j.$$

(i) Show that

$$\|x-\widehat{x}\|^2 = NM\left[\sum_{j\in S}|X_j-c_j|^2 + \sum_{j\notin S}|X_j|^2\right],$$

and conclude that for any fixed S, the best choice of coefficients is $c_j = X_j$.
Hint: Write $x-\widehat{x}$ in the DTFS basis and use orthogonality, as you did in part (e).

(ii) Now take $c_j = X_j$. Show that the resulting mean-squared error is

$$\text{MSE} = \frac{1}{NM}\|x-\widehat{x}\|^2 = \sum_{j\notin S}|X_j|^2,$$

i.e., exactly the energy contained in those indices you *didn't* keep.
Then recast this error as a sum of terms X_j for $j \in S$ to show that you can minimize MSE by picking the B *largest coefficients*.

Solution: **(i)** We can write both signals over the full index set and subtract:

$$x-\widehat{x} = \sum_{j=1}^{NM}(X_j-c_j)\,\psi_j.$$

Expanding the squared error as an inner product,

$$\begin{aligned}\|x-\widehat{x}\|^2 &= \langle x-\widehat{x}, x-\widehat{x}\rangle \\ &= \left\langle \sum_{j=1}^{NM}(X_j-c_j)\psi_j, \sum_{p=1}^{NM}(X_p-c_p)\psi_p \right\rangle \\ &= \sum_{j=1}^{NM}\sum_{p=1}^{NM}(X_j-c_j)(X_p-c_p)^*\langle\psi_j,\psi_p\rangle.\end{aligned}$$

By orthogonality, $\langle\psi_j,\psi_p\rangle = 0$ for $j \neq p$, so the cross terms vanish, and $\langle\psi_j,\psi_j\rangle = NM$. Hence,

$$\|x-\widehat{x}\|^2 = NM\sum_{j=1}^{NM}|X_j-c_j|^2.$$

We can split the sum into the indices we kept ($j \in S$) and the indices we discarded ($j \notin S$):

$$NM\sum_{j=1}^{NM}|X_j-c_j|^2 = NM\left[\sum_{j\in S}|X_j-c_j|^2 + \sum_{j\notin S}|X_j-c_j|^2\right].$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

But $c_j = 0$ for $j \notin S$, so $|X_j - c_j|^2 = |X_j|^2$, and we obtain the stated identity

$$\|x - \widehat{x}\|^2 = NM \left[\sum_{j \in S} |X_j - c_j|^2 + \sum_{j \notin S} |X_j|^2 \right].$$

We want to choose the best c_j that minimizes this error. We cannot do anything about the second sum, because there is no c_j. However, we *can* do something about the first sum. Since this is a sum of nonnegative terms, it is minimized by making it zero. Therefore we should choose $c_j = X_j$. In other words, the best approximation coefficients are exactly those found by DTFS.

(ii) With $c_j = X_j$ the error is $NM \sum_{j \notin S} |X_j|^2$. Dividing by NM gives MSE $= \sum_{j \notin S} |X_j|^2$.

Writing $E = \sum_{j=1}^{NM} |X_j|^2$ for the total energy, which does not depend on S, we can recast the error using a sum of terms *in* S:

$$\|x - \widehat{x}\|^2 = NM \left(E - \sum_{j \in S} |X_j|^2 \right)$$

Therefore, minimizing the error is the same as maximizing the retained energy $\sum_{j \in S} |X_j|^2$.

When is this sum the largest? This is precisely when we pick the largest possible X_js to keep. Hence, we should use the B largest values of $|X_j|$.

(h) Consider the checkerboard image x with $N = M = 32$ and its exact set of DTFS coefficients X. In this part, we will be reconstructing the image from a subset of its DTFS coefficients. Use each of the following three index-selection rules:

(i) Square set: keep indices with $k \leq \sqrt{B}$ and $\ell \leq \sqrt{B}$ for a coefficient budget of B.

(ii) Quarter circular set: keep indices with $k^2 + \ell_2 \leq R^2$.

(iii) Max set: keep the B largest magnitude coefficients.

For three budgets: 64, 256, and 576 **total** kept coefficients, reconstruct $\widehat{x}$ for each rule and compute the mean-squared error (MSE) in the image reconstruction. Provide the reconstructions for each rule and budget. Briefly discuss any visual reconstruction errors and compare the three rules. **Note**: For the quarter circular set, it's fine if the number of kept coefficients is slightly different from the budget.

Solution: The reconstruction quality improves as more coefficients are included. The max set has perfect approximation behavior at just 64 coefficients. This is because the checkerboard DTFS only has nonzero coefficients for 17 values of (k, ℓ). The square set and quarter circular set seem to perform similarly, but both display substantial edge effects at the boundaries of the image, due to lack of inclusion of the high-frequency coefficients (which allow for sharp edges).

2. Least Squares Proof

Let $\widehat{\mathbf{x}}$ be the approximate solution to a linear least squares problem.

$$\widehat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|\mathbf{b} - \mathbf{A}\mathbf{x}\|^2$$

Show that the minimizing least squares error vector $\boldsymbol{\varepsilon} = \mathbf{b} - \mathbf{A}\widehat{\mathbf{x}}$ is orthogonal to the columns of $\mathbf{A}$ by direct manipulation. Consider first the orthogonality condition for just one column, and then try to more compactly write the condition for *all* columns as a single matrix-vector multiplication.

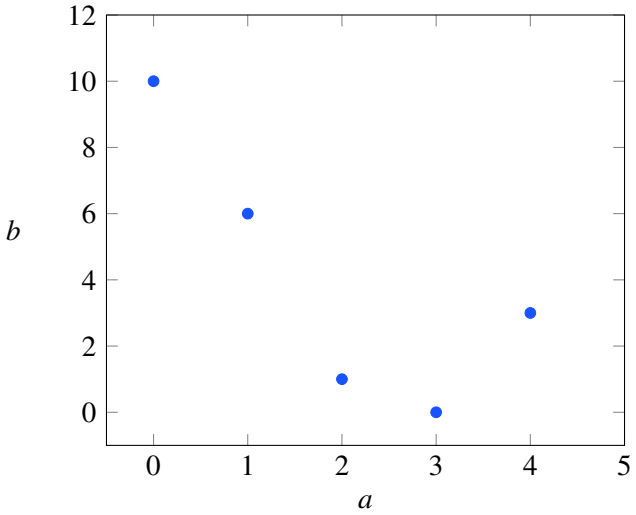
Solution:

We want to show that the error in the linear least squares estimate is orthogonal to the columns of the $\mathbf{A}$, i.e., we want to show that $\mathbf{A}^\top \boldsymbol{\varepsilon} = \mathbf{A}^\top (\mathbf{b} - \mathbf{A}\widehat{\mathbf{x}})$ is the zero vector. Plugging in the linear least squares formula for $\widehat{\mathbf{x}}$, we get

$$\begin{aligned}
\mathbf{A}^\top \boldsymbol{\varepsilon} = \mathbf{A}^\top (\mathbf{b} - \mathbf{A}\widehat{\mathbf{x}}) &= \mathbf{A}^\top \left(\mathbf{b} - \mathbf{A} \left(\mathbf{A}^\top \mathbf{A} \right)^{-1} \mathbf{A}^\top \mathbf{b} \right) \\
&= \mathbf{A}^\top \mathbf{b} - \mathbf{A}^\top \mathbf{A} \left(\mathbf{A}^\top \mathbf{A} \right)^{-1} \mathbf{A}^\top \mathbf{b} \\
&= \mathbf{A}^\top \mathbf{b} - \mathbf{I}\mathbf{A}^\top \mathbf{b} \\
&= \mathbf{A}^\top \mathbf{b} - \mathbf{A}^\top \mathbf{b} = \mathbf{0}
\end{aligned}$$

3. Picking a Model for Least Squares

One of the most challenging aspects of performing least squares is *choosing the right model to fit*. Ultimately, most of the math will be done by a computer, but picking the right model is the engineer's job. In this problem we will see what happens if you pick the right, or the wrong, model. Suppose we've collected the following datapoints:



a	0	1	2	3	4
b	10	6	1	0	3

(a) Consider the above data points. Find the constant model of the form

$$\mathbf{b} = x\mathbf{1}$$

that best fits the data, i.e. find the scalar value of $x = \hat{x}$ that minimizes the squared error

$$\left\|\mathbf{b} - \widehat{\mathbf{b}}\right\|^2 = \|\mathbf{b} - x\mathbf{1}\|^2 = \left\| \begin{bmatrix} b_1 \\ \vdots \\ b_5 \end{bmatrix} - \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} x \right\|^2. \tag{1}$$

You should set up a least squares problem and solve it, either by hand or with a computer, in this part. Then, sketch approximately or plot in python your fitted model as well as the 5 data points. Also compute the total squared error. Finally, interpret the solution you found. Put yourself in the shoes of someone who has *never heard* of least squares: is this an intuitive way to fit a "dumb" model that always predicts a constant?

Solution:

With $\mathbf{A} = \begin{bmatrix}1 & 1 & 1 & 1 & 1\end{bmatrix}^T$ and $\mathbf{b} = \begin{bmatrix}10 & 6 & 1 & 0 & 3\end{bmatrix}^T$, we apply the linear least squares formula

to find

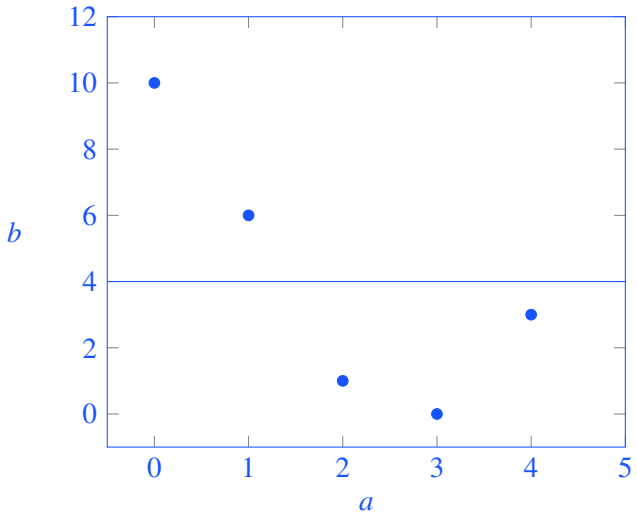
$$\begin{aligned}\hat{x} &= (\mathbf{1}^T\mathbf{1})^{-1}\mathbf{1}^T\mathbf{b} \\ &= \left(\begin{bmatrix}1 & 1 & 1 & 1 & 1\end{bmatrix}\begin{bmatrix}1\\1\\1\\1\\1\end{bmatrix}\right)^{-1}\begin{bmatrix}1 & 1 & 1 & 1 & 1\end{bmatrix}\begin{bmatrix}10\\6\\1\\0\\3\end{bmatrix} \\ &= (5)^{-1}(20) = 4.\end{aligned}$$

The error between the model's prediction and actual b values is

$$\begin{aligned}\mathbf{b} - \widehat{\mathbf{b}} &= \mathbf{b} - \hat{x}\mathbf{1} \\ &= \begin{bmatrix}10\\6\\1\\0\\3\end{bmatrix} - 4\begin{bmatrix}1\\1\\1\\1\\1\end{bmatrix} = \begin{bmatrix}6\\2\\-3\\-4\\-1\end{bmatrix},\end{aligned}$$

and the squared error is

$$\left\|\mathbf{b} - \widehat{\mathbf{b}}\right\|^2 = 66.$$



Performing least squares on this model guesses the average value of the data points. This is a pretty intuitive choice, for such an inflexible model! If you'd never heard of least squares, you might be able to fit a constant model. However, a constant doesn't predict any of the parabola-esque shape of the data, which is reflected in the large squared error. We address this in the next part.

(b) Look back at the original scatter plot of the data. For this part, *you* should pick a model which appears to closely fit the given points. Set up the problem, solve it, sketch or plot your model next to the scattered data, and report squared error. Then, explain why it is important to pick the right model.

Solution:

Looking at the scatter plot, we see a U-shape that reminds us of a parabola. Hence, we try to fit a quadratic model

$$\mathbf{b} = x_1\mathbf{a}^2 + x_2\mathbf{a} + x_3\mathbf{1},$$

or equivalently $\mathbf{b} \approx \mathbf{A}\mathbf{x}$ with

$$\mathbf{A} = \begin{bmatrix} a_1^2 & a_1 & 1 \\ a_2^2 & a_2 & 1 \\ a_3^2 & a_3 & 1 \\ a_4^2 & a_4 & 1 \\ a_5^2 & a_5 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \\ 4 & 2 & 1 \\ 9 & 3 & 1 \\ 16 & 4 & 1 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 10 \\ 6 \\ 1 \\ 0 \\ 3 \end{bmatrix}.$$

The least-squares estimate is $\widehat{\mathbf{x}} = \left(\mathbf{A}^T\mathbf{A}\right)^{-1}\mathbf{A}^T\mathbf{b}$. Written out explicitly,

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 354 & 100 & 30 \\ 100 & 30 & 10 \\ 30 & 10 & 5 \end{bmatrix}, \qquad \mathbf{A}^T\mathbf{b} = \begin{bmatrix} 58 \\ 20 \\ 20 \end{bmatrix}.$$

Solving the 3×3 system (by calculator or computer) gives

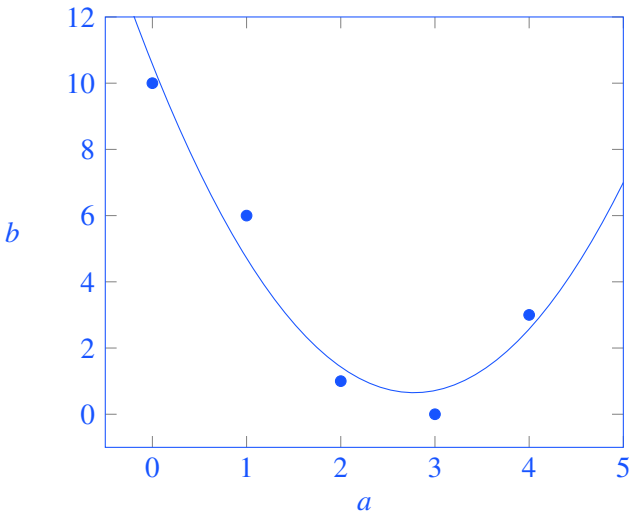
$$\widehat{\mathbf{x}} = \begin{bmatrix} \hat{x}_1 \\ \hat{x}_2 \\ \hat{x}_3 \end{bmatrix} = \begin{bmatrix} 9/7 \\ -50/7 \\ 74/7 \end{bmatrix} \approx \begin{bmatrix} 1.29 \\ -7.14 \\ 10.57 \end{bmatrix}.$$

The fitted model is therefore

$$\hat{b}(a) = \frac{9}{7}a^2 - \frac{50}{7}a + \frac{74}{7}.$$

Evaluating at the five data a-values,

$$\widehat{\mathbf{b}} = \begin{bmatrix} 74/7 \\ 33/7 \\ 10/7 \\ 5/7 \\ 18/7 \end{bmatrix} \approx \begin{bmatrix} 10.57 \\ 4.71 \\ 1.43 \\ 0.71 \\ 2.57 \end{bmatrix}, \qquad \mathbf{b} - \widehat{\mathbf{b}} = \begin{bmatrix} -4/7 \\ 9/7 \\ -3/7 \\ -5/7 \\ 3/7 \end{bmatrix}, \qquad \left\|\mathbf{b} - \widehat{\mathbf{b}}\right\|^2 = \frac{20}{7} \approx 2.86.$$



Compare this with part (a). Even though we used least-squares to fit both models, we got drastically different errors: 66 when we chose to model with a constant, versus about 2.86 when we used a parabola. Take away from this problem that a computer will happily fit whichever model we hand it, and that it is up to you as an engineer to choose a model so as to make least squares *effective*.

UCB EE 66, Fall 2026, Homework 6, All Rights Reserved. This may not be publicly shared without explicit permission.

4. Shape of Cells

Suppose you would like to count the number of cells in a given biological sample. However, you only have access to pixels from a digital image of the cells! To successfully complete your task, you have to somehow determine the count and locations of cells using only the pixel data. This requires you to automatically detect all the cell-like shapes in our image, which happens to be an incredibly widespread and relevant problem! As an example, consider a robot trying to pinpoint where a mug is in its field of view.

Anyhoo, let's return to our medical imaging scenario, but simplified. You are interested in finding the exact position and shape of **one** cell in an image. You will do this by finding the **equation of the circle or ellipse** that bounds the cell relative to a given coordinate system.

Your collaborator uses edge detection techniques[1] to find **a bunch of points that are approximately along the edge of a cell**. Your collaborator is using standard (x,y) coordinate axes, with the origin of the coordinate system placed at the center of the image. They give you the following coordinates for points that approximately bound the cell:

$(0.3,-0.69)$, $(0.5,0.87)$, $(0.9,-0.86)$, $(1,0.88)$, $(1.2,-0.82)$, $(1.5,0.64)$, $(1.8,0)$.

Recall that an equation of the form

$$a_1x^2+b_1xy+c_1y^2+d_1x+e_1y=1$$

can be used to represent an ellipse (if $b_1^2-4a_1c_1<0$),
and an equation of the form

$$a_1(x^2+y^2)+d_1x+e_1y=1$$

is a circle if $d_1^2+e_1^2+4a_1>0$. Notice that the circle has fewer parameters.

You don't need to consider these constraints in your setup, but you are encouraged to check whether your solutions satisfy these constraints.

(a) How can you find the equation of a *circle* that surrounds the cell by fitting the data points? First, formulate a minimization problem to do this, i.e. a least squares problem minimizing the squared error $\|\mathbf{A}\mathbf{x}_c-\mathbf{b}\|^2$, where you attempt to find the **unknown coefficients** a_1**,** d_1**, and** e_1 from your data points. Here your unknown vector $\mathbf{x}_c=\begin{bmatrix}a_1\\d_1\\e_1\end{bmatrix}$.

Hint: The quantities (x^2+y^2), x, and y can be thought of as known values calculated from your data points.

You do not need to simplify the numerical values of the matrix elements; just writing out the data matrix symbolically will suffice. **Solution:** For a circle, we know our equation is:

$$a_1(x^2+y^2)+d_1x+e_1y=1$$

Note that $\mathbf{b}$ corresponds to the 1 in our equation, and we can use our coefficeints d_1,e_1, and a_1 as our vector of unknowns $(\mathbf{x}_c)$.
So writing this equation out for a single set of points, can be thought of as:

$$\begin{bmatrix}x^2+y^2 & x & y\end{bmatrix}\begin{bmatrix}a_1\\d_1\\e_1\end{bmatrix}-\begin{bmatrix}1\end{bmatrix}=0$$

[1] If you're interested in learning more about these, take CS180 in the future!

Extending this for multiple sets of points x, y, we can repeat this row to create our least squares setup:

$$\min_{a_1,d_1,e_1} \left\| \begin{bmatrix} x^2+y^2 & x & y \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} a_1 \\ d_1 \\ e_1 \end{bmatrix} - \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} \right\|$$

(b) How can you find the equation of an ellipse (instead of a circle) that surrounds the cell? Formulate a minimization problem similar to that in part (a) with the unknown vector defined as $\mathbf{x}_e$. Note that $\mathbf{x}_e$ will be a different vector than $\mathbf{x}_c$.

You do not need to simplify the numerical values of the matrix elements; just writing out the data matrix symbolically will suffice.

Solution:

In a similar fashion to part (a), the setup is:

$$\min_{a_1,b_1,c_1,d_1,e_1} \left\| \begin{bmatrix} x^2 & xy & y^2 & x & y \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} a_1 \\ b_1 \\ c_1 \\ d_1 \\ e_1 \end{bmatrix} - \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} \right\|$$

(c) Now let's try this ourselves with the data given in the problem. **In the iPython notebook**, write a short program that uses least-squares to fit a circle to the given points. A helper function `plot_circle` is provided. What is $\frac{\|\mathbf{e}\|}{N}$, where $\mathbf{e} = \mathbf{A}\mathbf{x}_c - \mathbf{b}$ and N is the number of data points? Plot your points and the best fit circle in IPython.

Solution:

See the IPython notebook.

The solution vector is:

$$\mathbf{x}_c = \begin{bmatrix} 4.87 \\ -7.89 \\ -0.23 \end{bmatrix}.$$

Thus, we would predict the equation of the circle to be: $4.87(x^2+y^2) - 7.89x - 0.23y = 1$.

This gives the normalized error: $\frac{0.96}{7} = 0.137$.

(d) **In the iPython notebook**, write a short program that uses least-squares to fit an ellipse to the given points. A helper function `plot_ellipse` is provided. What is $\frac{\|\mathbf{e}\|}{N}$, where $\mathbf{e} = \mathbf{A}\mathbf{x}_e - \mathbf{b}$ and N is the number of data points? Plot your points and the best fit ellipse in IPython. How does this error compare to the one in the previous subpart?

Solution:

See the IPython notebook.

The solution vector is:

$$\mathbf{x}_e = \begin{bmatrix} 4.10 \\ 0.49 \\ 4.94 \\ -6.85 \\ -0.62 \end{bmatrix}.$$

We predict the general equation to be: $4.10x^2 + 0.49xy + 4.94y^2 - 6.85x - 0.62y = 1$.

This gives the normalized error: $\frac{0.090}{7} = 0.0128$.

The error is less than the circle fit, as predicted.

5. Homework Reflection

Doing homework is an **investment in your education**, and the course staff wants to hear about your experience with this problem set! After finishing the rest of the homework, fill out these brief reflections on what you got out of your time. We'll read this and take your answers into account as we write future assignments, in this semester and future ones.

Note that we are *genuinely curious* to read what you have to write. We expect *brief but thoughtful* responses. **Hence, this part will be weighed significantly in your homework completion grade.**

(a) Did you work on this homework alone or in a group? Did this work well for you? *We encourage you to attempt each problem on your own first, and to work with a group when you're stuck.*

Solution:

I worked alone because my study group was out of town.

I prefer to work alone because a group can be very distracting for me.

I worked with my friends Alice and Bob. It's nice to be able to bounce ideas off one another and fill in each other's gaps. Explaining my ideas to someone else is hard, but it really makes sure I understand them well!

I met some new friends Carol and Danny at homework party, and we were able to finish most of the homework together with occasional help from a tutor. I reviewed my answers on my own to make sure I understood everything.

(b) What resources did you use to solve the homework problems? How did you use these resources, and how did they help you? Feel free to mention LLM tools, if you've used any.

Solution:

I didn't remember some of the definitions I learned in linear algebra, so I consulted my old notes and reviewed concepts as I needed to while solving the problems

I went to office hours to work on the homework, and whenever I got stuck, I asked one of the tutors to provide some hints or review concepts I didn't remember.

I tried all the problems on my own, and then asked an LLM to critique my approach. I got 90% of the way there on my own, but this interaction enriched my understanding of the material!

I reviewed my lecture notes before starting the homework; and whenever I got stuck, I scoured the discussion worksheets for similar problems and used the solutions to guide my thought process.

I asked some questions on Ed whenever I was confused so that I could get the most fitting answer to my question, in the context of EE66. I also answered some questions because this is a good way to practice the stuff I know well.

I couldn't even understand what problem 2 was asking, so I asked an AI tool to review all the relevant prerequisite knowledge and explain the problem statement in a different way before trying to tackle it on my own.

(c) What was the most fun / interesting problem on this homework? What was the least fun / un-interesting problem?

Solution:

I found question 3 very intriguing because this is a totally different perspective on DTFS to what we learned in lecture. Question 2 was pretty boring and mechanical, because we were just practicing the synthesis equation, but I suppose it is important to be able to do these more-basic things too.

(d) Did you feel prepared to tackle these problems? Were there any problems that seemed completely disjoint from what you've seen in class? Was there anything covered in class that you weren't able practice in this assignment?

Solution: I did feel prepared! But, question 5 seemed to me to be out of place because it's been two weeks since we covered least squares. We talked about SVD and its application for PCA in class and discussion this week, and I didn't really understand any of it, so it would have been nice to have some problems which gave me an opportunity to practice those concepts.

(e) Is there anything else you would like to share?

Solution:

Johnny, who is a tutor, was SUPER DUPER helpful in office hours. He explained stuff in such a way that I still felt like I was discovering the answer to the problems by myself, which was awesome.

I'm feeling very overwhelmed by this class and the upcoming midterm is really stressing me out. These homeworks are really hard and I wish there were more resources available, which I could use to understand them deeply.